Decodificador de 7 segmentos

Taller de segundo corte

Sebastian Bejarano – Universidad de San Buenaventura

Repositorio: https://github.com/<usuario>/<repositorio>

1. Diseño de los patrones

La tabla tiene 16 entradas (4 bits de entrada, 7 bits de salida) y sigue el orden de segmentos a, b, c, d, e, f, g, con a como el bit de la izquierda. Un 1 enciende el segmento. El archivo `tabla.txt` tiene 16 líneas de 7 bits, que es el formato que lee `read_table`.

Entrada	abcdefg	Figura
0	`1111110`	0
1	`0110000`	1 a la derecha (segmentos b y c)
2	`1101101`	2
3	`1111001`	3
4	`0110011`	4
5	`1011011`	5
6	`1011111`	6 cerrado (enciende el segmento superior a)
7	`1110000`	7
8	`1111111`	8
9	`1111011`	9 cerrado (enciende el segmento inferior d)
10	`0000001`	signo menos
11	`0110110`	dos barras verticales
12	`1100011`	cuadro superior
13	`0011101`	cuadro inferior
14	`1001001`	tres rayas horizontales
15	`0011100`	u minúscula

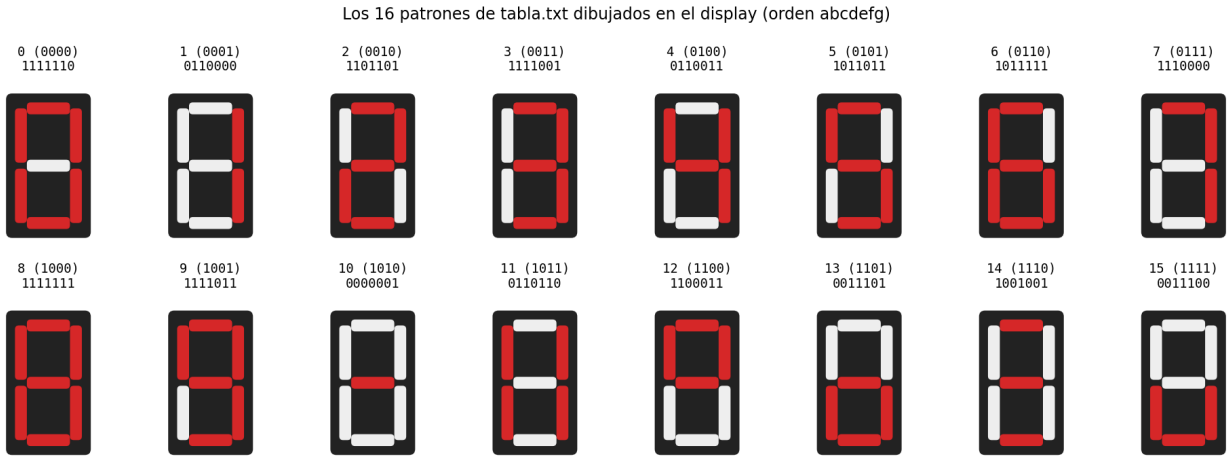


Figura 1. Los 16 patrones de la tabla dibujados en el display.

Decisiones de diseño

- **El 1** se dibuja a la derecha, con los segmentos b y c.
- **El 6 y el 9 son cerrados.** El 6 enciende el segmento superior (a) y el 9 enciende el inferior (d). Así no se confunden con una b o una q vistos de lejos.
- **Entradas 10 a 15.** No uso las letras A a F. Las figuras son: signo menos (10), que sirve para marcar un número negativo; dos barras verticales (11), parecidas al símbolo de pausa; cuadro superior (12), para valor alto o desbordamiento; cuadro inferior (13), para valor bajo; tres rayas (14), para "procesando"; y u minúscula (15), que se puede leer como "usuario" o "unidad". Ninguna repite un patrón de 0 a 9.

2. Comandos utilizados

Dependencias:

```
pip install myhdl pytest
```

Simulación (genera el archivo VCD) y conversión a VHDL, el lenguaje por defecto:

```
python sevensegdec.py --simulation --table tabla.txt --outdir salida
```

Conversión a Verilog:

```
python sevensegdec.py --table tabla.txt --verilog --outdir salida
```

Pruebas unitarias:

```
pytest -v
```

3. Resultados de la simulación

```
Salida real de la simulación (VHDL)

$ python sevensegdec.py --simulation --table tabla.txt --outdir salida
OK:    en (10) entrada bcd  0 (0000) -> sseg = 1111110
OK:    en (20) entrada bcd  1 (0001) -> sseg = 0110000
OK:    en (30) entrada bcd  2 (0010) -> sseg = 1101101
OK:    en (40) entrada bcd  3 (0011) -> sseg = 1111001
OK:    en (50) entrada bcd  4 (0100) -> sseg = 0110011
OK:    en (60) entrada bcd  5 (0101) -> sseg = 1011011
OK:    en (70) entrada bcd  6 (0110) -> sseg = 1011111
OK:    en (80) entrada bcd  7 (0111) -> sseg = 1110000
OK:    en (90) entrada bcd  8 (1000) -> sseg = 1111111
OK:    en (100) entrada bcd  9 (1001) -> sseg = 1111011
OK:    en (110) entrada bcd 10 (1010) -> sseg = 0000001
OK:    en (120) entrada bcd 11 (1011) -> sseg = 0110110
OK:    en (130) entrada bcd 12 (1100) -> sseg = 1100011
OK:    en (140) entrada bcd 13 (1101) -> sseg = 0011101
OK:    en (150) entrada bcd 14 (1110) -> sseg = 1001001
OK:    en (160) entrada bcd 15 (1111) -> sseg = 0011100
<class 'myhdl.StopSimulation'>: No more events
Simulación terminada: 16 casos, 0 errores.
Conversión terminada (VHDL) en 'salida'.
```

Figura 2. Salida de la simulación: las 16 entradas con la salida leída en cada una.

La línea `No more events` es el aviso normal de MyHDL cuando ya no quedan eventos por procesar. No es un error.

Qué verifica el testbench

Recorre las 16 entradas posibles, de 0000 a 1111. Después de cada una espera 10 ns, lee la señal de salida y la compara con la línea que le corresponde en la tabla. Si encuentra una diferencia, imprime un ERROR con el valor esperado y el leído.

Por qué no aparece ningún ERROR

El decodificador y el testbench usan la misma tabla. Si la LUT está bien conectada y cada entrada llega a su posición, las dos cosas coinciden siempre. Por eso este chequeo comprueba la mecánica del circuito (indexación, conexión de señales, tiempos), pero no detectaría una tabla mal diseñada ni un error de digitación. Esa parte la cubren las pruebas con pytest, donde los valores esperados se escriben aparte.

Qué representan los 7 bits de sseg

Cada bit controla un segmento del display, de izquierda a derecha: a, b, c, d, e, f, g. Un 1 enciende el segmento. Por ejemplo, 1111110 enciende de la a a la f y deja apagado el g, que es el 0.

Interpretación del VCD

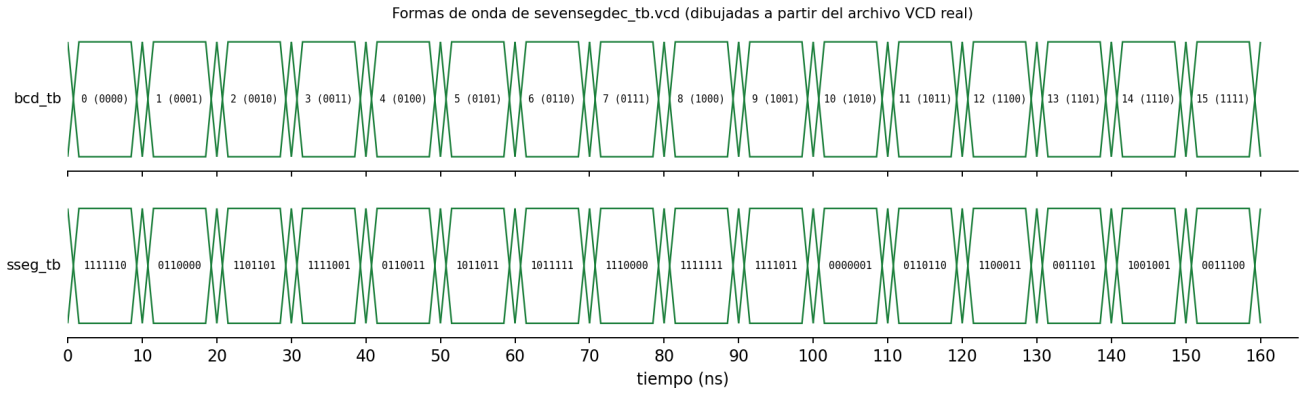


Figura 3. Formas de onda del archivo sevensegdec_tb.vcd, dibujadas a partir de ese mismo archivo.

- La entrada (bcd_tb) sube en uno cada 10 ns: empieza en 0 y llega a 15 a los 150 ns. La simulación termina a los 160 ns.
- La salida (sseg_tb) cambia en el mismo instante que la entrada, sin retardo visible. Es lo esperado en lógica combinacional: no hay reloj ni registros, así que la salida depende solo de la entrada del momento.
- Cada valor de sseg_tb es el de la tabla, desde 1111110 para el 0 hasta 0011100 para el 15.

Código HDL generado

Los archivos generados están en la carpeta `salida` del repositorio: `sevensegdec_nexys.vhd` (junto con `pck_myhdl_011.vhd`, que necesita) y `sevensegdec_nexys.v` en Verilog.

La tabla de Python se convierte en una instrucción case dentro de un proceso sensible a la entrada bcd (en Verilog, un case dentro de un always @(bcd)). Cada línea de la tabla pasa a ser una rama: para la entrada 0 se asigna 1111110, para la 1 se asigna 0110000, y así sucesivamente. La rama del 15 queda escrita como "others", porque MyHDL usa la última rama para cubrir todo lo que falta. No hay reloj ni flip-flops, así

que es lógica puramente combinacional: en una FPGA la herramienta de síntesis la implementa con LUTs, y cada bit de sseg es una función lógica de los 4 bits de entrada.

El archivo también incluye lo que agrega la envoltura para la Nexys4 DDR: un proceso que invierte la salida cuando el puerto invert vale 1 (los segmentos de esa tarjeta son activos en bajo), la señal bcdled, que repite la entrada en los LEDs, y ssanodes, fijo en 11111110 para que solo se encienda el primer display.

4. Pruebas unitarias con pytest

Las pruebas están en `test_sevensegdec.py`, separadas del script, que solo conserva la lógica. Son 58 pruebas:

- 16 casos parametrizados que comparan tabla.txt con los valores esperados escritos en la propia prueba.
- 16 casos parametrizados que simulan el decodificador en MyHDL y comparan la salida.
- 16 casos más con la salida invertida.
- Propiedades de la tabla: 16 patrones distintos, ninguno apagado, el 1 a la derecha, el 6 y el 9 cerrados.
- Lectura de la tabla: se rechazan archivos con menos de 16 líneas, con ancho distinto de 7 o con caracteres que no son 0 ni 1.
- El testbench del script termina con cero errores, y la conversión a VHDL y a Verilog genera los archivos con el case esperado.

Para comprobar que las pruebas sirven, apagué a propósito el segmento g del 6 en una copia temporal de la tabla. Fallaron 3 pruebas, todas del caso 6: la de la tabla, la del decodificador y la del decodificador invertido.

Cambios respecto al script del curso

- read_table valida el archivo y lanza un error claro. Antes devolvía una tabla de ceros sin avisar.
- El testbench cuenta los errores y muestra el resultado de cada caso.
- Con la opción de inversión, el testbench original marcaba error en las 16 entradas porque comparaba la salida negada contra la tabla sin negar. Ahora compara contra el valor negado.
- Se agregó la opción para elegir la carpeta de salida del VCD y del HDL.

Anexo A. Resultado de las pruebas (pytest)

```
Salida real de pytest (58 pruebas)

$ pytest -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/taller-7seg
collecting ... collected 58 items

test_sevensegdec.py::test_tabla_archivo[bcd=0] PASSED                    [  1%]
test_sevensegdec.py::test_tabla_archivo[bcd=1] PASSED                    [  3%]
test_sevensegdec.py::test_tabla_archivo[bcd=2] PASSED                    [  5%]
test_sevensegdec.py::test_tabla_archivo[bcd=3] PASSED                    [  6%]
test_sevensegdec.py::test_tabla_archivo[bcd=4] PASSED                    [  8%]
test_sevensegdec.py::test_tabla_archivo[bcd=5] PASSED                    [ 10%]
test_sevensegdec.py::test_tabla_archivo[bcd=6] PASSED                    [ 12%]
test_sevensegdec.py::test_tabla_archivo[bcd=7] PASSED                    [ 13%]
test_sevensegdec.py::test_tabla_archivo[bcd=8] PASSED                    [ 15%]
test_sevensegdec.py::test_tabla_archivo[bcd=9] PASSED                    [ 17%]
test_sevensegdec.py::test_tabla_archivo[bcd=10] PASSED                   [ 18%]
test_sevensegdec.py::test_tabla_archivo[bcd=11] PASSED                   [ 20%]
test_sevensegdec.py::test_tabla_archivo[bcd=12] PASSED                   [ 22%]
test_sevensegdec.py::test_tabla_archivo[bcd=13] PASSED                   [ 24%]
test_sevensegdec.py::test_tabla_archivo[bcd=14] PASSED                   [ 25%]
test_sevensegdec.py::test_tabla_archivo[bcd=15] PASSED                   [ 27%]
test_sevensegdec.py::test_decodificador[bcd=0] PASSED                    [ 29%]
test_sevensegdec.py::test_decodificador[bcd=1] PASSED                    [ 31%]
test_sevensegdec.py::test_decodificador[bcd=2] PASSED                    [ 32%]
test_sevensegdec.py::test_decodificador[bcd=3] PASSED                    [ 34%]
test_sevensegdec.py::test_decodificador[bcd=4] PASSED                    [ 36%]
test_sevensegdec.py::test_decodificador[bcd=5] PASSED                    [ 37%]
test_sevensegdec.py::test_decodificador[bcd=6] PASSED                    [ 39%]
test_sevensegdec.py::test_decodificador[bcd=7] PASSED                    [ 41%]
test_sevensegdec.py::test_decodificador[bcd=8] PASSED                    [ 43%]
test_sevensegdec.py::test_decodificador[bcd=9] PASSED                    [ 44%]
test_sevensegdec.py::test_decodificador[bcd=10] PASSED                   [ 46%]
test_sevensegdec.py::test_decodificador[bcd=11] PASSED                   [ 48%]
test_sevensegdec.py::test_decodificador[bcd=12] PASSED                   [ 50%]
test_sevensegdec.py::test_decodificador[bcd=13] PASSED                   [ 51%]
test_sevensegdec.py::test_decodificador[bcd=14] PASSED                   [ 53%]
test_sevensegdec.py::test_decodificador[bcd=15] PASSED                   [ 55%]
test_sevensegdec.py::test_decodificador_invertido[bcd=0] PASSED          [ 56%]
test_sevensegdec.py::test_decodificador_invertido[bcd=1] PASSED          [ 58%]
test_sevensegdec.py::test_decodificador_invertido[bcd=2] PASSED          [ 60%]
test_sevensegdec.py::test_decodificador_invertido[bcd=3] PASSED          [ 62%]
test_sevensegdec.py::test_decodificador_invertido[bcd=4] PASSED          [ 63%]
test_sevensegdec.py::test_decodificador_invertido[bcd=5] PASSED          [ 65%]
test_sevensegdec.py::test_decodificador_invertido[bcd=6] PASSED          [ 67%]
test_sevensegdec.py::test_decodificador_invertido[bcd=7] PASSED          [ 68%]
test_sevensegdec.py::test_decodificador_invertido[bcd=8] PASSED          [ 70%]
test_sevensegdec.py::test_decodificador_invertido[bcd=9] PASSED          [ 72%]
test_sevensegdec.py::test_decodificador_invertido[bcd=10] PASSED         [ 74%]
test_sevensegdec.py::test_decodificador_invertido[bcd=11] PASSED         [ 75%]
test_sevensegdec.py::test_decodificador_invertido[bcd=12] PASSED         [ 77%]
test_sevensegdec.py::test_decodificador_invertido[bcd=13] PASSED         [ 79%]
test_sevensegdec.py::test_decodificador_invertido[bcd=14] PASSED         [ 81%]
test_sevensegdec.py::test_decodificador_invertido[bcd=15] PASSED         [ 82%]
test_sevensegdec.py::test_16_patrones_distintos PASSED                   [ 84%]
test_sevensegdec.py::test_ningun_patron_apagado PASSED                   [ 86%]
test_sevensegdec.py::test_decisiones_de_diseno PASSED                    [ 87%]
test_sevensegdec.py::test_read_table_acepta_linea_en_blanco_final PASSED [ 89%]
test_sevensegdec.py::test_read_table_rechaza_menos_de_16_lineas PASSED   [ 91%]
test_sevensegdec.py::test_read_table_rechaza_ancho_incorrecto PASSED     [ 93%]
test_sevensegdec.py::test_read_table_rechaza_caracteres_no_binarios PASSED [ 94%]
test_sevensegdec.py::test_testbench_del_script_sin_errores PASSED        [ 96%]
test_sevensegdec.py::test_conversion_a_hdl[vhdl] PASSED                  [ 98%]
test_sevensegdec.py::test_conversion_a_hdl[verilog] PASSED               [100%]

============================== 58 passed in 0.50s ==============================
```